https://pdfkit.org/ for more deep dive

1. الـ `PDFDocument` (الماكينة)

JavaScript

```
const doc = new PDFDocument();
```

- **المفهوم:** ده "العقل المدبر". الـ Class ده هو اللي بيكريه Instance جديد من الملف في الميموري.
- **الزتونة:** أول ما بتنادي `new PDFDocument()` ، بيبدأ يجهز الـ Buffer والـ Metadata بتاعة الملف. هو لسه مش ملف على الهارد، هو عبارة عن Object تايه في الرام مستني توصله بـ "ماسورة" (Stream).

---

2. الـ `doc.pipe(output)` (الوصلة)

- **المفهوم:** الـ PDFKit مبني بالكامل فوق الـ **Node.js Streams**.
- **التحليل:** `doc` هو **Readable Stream**. عشان تطلع منه ملف، لازم توصله بـ **Writable Stream**.
  - لو عايزه ملف على الهارد: بتوصله بـ `fs.createWriteStream` .
  - لو عايزه يتبعت فوراً لليوزر (Response): بتوصله بـ `res` .
- **القوة:** الداتا بتخرج من الماكينة للمتصفح "أول بأول" (Real-time)، وده بيخلي السيرفر ميتخنقش لو الملف 1000 صفحة.

---

3. الـ Coordinate System (نظام الإحداثيات)

الـ PDF مش زي الـ HTML بيمشي بالـ Flow العادي؛ الـ PDF بيمشي بالـ **Points**.

- **المفهوم:** النقطة (Point) بتساوي 1/72 بوصة.
- **الإحداثيات:** الـ (0,0) هي "الركن الشمال فوق".
- **التحليل:** لما بتقول `doc.text('Hello', 100, 100)` ، إنت بتقوله "روح عند النقطة 100 عرض و100 طول واكتب". إنت "المهندس" اللي بيحدد مكان كل ملّي في الصفحة.

---

4. الـ Vector Graphics (الرسم الهندسي)

الـ PDFKit في قلبه محرك رسومات (Vector Graphics Engine).

- **المفهوم:** إنت مش بتحط بكسلات، إنت بتدي أوامر رياضية (زي الـ SVG).

- **الأوامر:**
  - `moveTo(x, y)` : ارفع القلم وحطه هنا.
  - `lineTo(x, y)` : اسحب خط للنقطة دي.
  - `stroke()` : ارسم الخط فعلياً.
- **النتيجة:** الملف مهما عملت فيه Zoom، الجودة بتفضل "مسطرة" لأنها معادلات رياضية مش بكسلات.

---

5. الـ Method Chaining (رص الأوامر)

المكتبة دي بتحب الـ Chaining جداً:

JavaScript

```
doc.fontSize(25).fillColor('red').text('Hello World!');
```

- **التحليل:** كل ميثود بترجع الـ `doc` تاني. ده بيخلي الكود "Clean" وبيخليك تغير الإعدادات (اللون، الفونت، الحجم) قبل ما تنفذ أمر الكتابة الفعلي.

---

6. الـ Images و الـ Text Wrapping (التغليف)

الـ PDFKit ذكي في التعامل مع النصوص الطويلة:

- **Automatic Wrapping:** لو كتبت نص طويل، هو بيفهم أبعاد الصفحة أوتوماتيك وينزل سطر جديد.
- **Images:** جوه الملف. تقدر تتحكم في الـ **Embedding** لما بتضيف صورة، المكتبة بتعمل لها `width` والـ `height` أو `fit` تعمل لها جوه برواز معين.

---

7. الـ `doc.end()` (كلمة النهاية)

- **المفهوم:** دي أهم "تكة" في الكود.
- **التحليل:** الـ Stream بيفضل مفتوح ومستعد يستقبل داتا للأبد. الـ `doc.end()` بتقول للمكتبة: "خلاص، قفلي الملف، اكتبي الـ Footer بتاع الـ PDF، واقفل الـ Stream".
- **ملحوظة:** لو نسيت السطر ده، الملف هيفضل "متعلق" في السيرفر ومش هيوصل لليوزر أبداً، والرامات هتفضل محجوزة.

أولاً: الـ Vector Graphics (الرسم بالرياضة مش بالبكسل)

الـ PDFKit مبيتعاملش مع الصور كـ "بكسلات" بتبكسل لما تكبر، هو بيتعامل مع **Paths** (مسارات هندسية).

1. **الـ Pen Movements (حركة القلم):**

- `moveTo(x, y)` : دي معناها "ارفع القلم وحطه عند النقطة دي" من غير ما ترسم خط.
- `lineTo(x, y)` : دي معناها "اسحب خط من مكانك الحالي لحد النقطة دي".
  - **الزتونة:** أي شكل في الدنيا هو عبارة عن سلسلة من الـ `lineTo` وفي الآخر بتقفلها بـ `closePath()` .

2. **الأشكال الجاهزة (Built-in Shapes):**
   - بدل ما تقعد ترسم أربع خطوط عشان تعمل مستطيل، عندك `rect(x, y, width, height)` .
   - عندك كمان `ellipse` , `circle` , و `polygon` .
   - **تريكة صايعة:** الـ `roundedRect` بتخليك تعمل مستطيل بحواف ناعمة، وده بيدي شكل "مودرن" للجداول.

3. **الـ Filling & Stroking (التلوين والبرواز):**

- `fill()` : بتلون المساحة اللي جوه الشكل.
- `stroke()` : بترسم "البرواز" أو الخط الخارجي بس.
- `fillAndStroke()` : بتعمل الاتنين مع بعض.
  - **الـ Opacity:** تقدر تخلي الرسمة "شفافة" عن طريق `fillOpacity(0.5)` .

4. **الـ SVG Paths (ليفل الوحش):**

- أقوى ميزة هي ميثود `path('M 0 0 L 10 10...')` . دي بتخليك تاخد أي كود رسم من ملف **SVG** (زي لوجو الشركة مثلاً) وترسمه جوه الـ PDF فوراً بنفس الجودة.

ثانياً: الـ Paper Sizes (مقاس الرقعة)

لو إنت فاكر إن الـ PDF هو ورقة A4 وبس، تبقى غلطان. PDFKit بيديك تحكم كامل في "اللوحة" اللي بترسم عليها.

1. **الـ Presets (المقاسات الجاهزة):**
   - المكتبة بتدعم كل المقاسات العالمية: `A4` (المشهور في مصر)، `Letter` (المشهور في أمريكا)، `Legal` , و `Tabloid` .
   - **الكود:** `const doc = new PDFDocument({ size: 'A4' });`

2. **الـ Custom Sizes (المقاسات التفصيل):**
   - لو بتعمل "وصل أمانة" أو "فاتورة كاشير" صغيرة، مش هتستخدم A4.
   - تقدر تبعت Array فيه المقاس بالـ **Points** (الـ بوصة = 72 نقطة):

- `size: [400, 600]` (العرض 400، الطول 600).

3. **الـ Orientation (العدلة):**

- **Portrait:** الورقة بالطول (الديفولت).
- **Landscape:** الورقة بالعرض.
  - **التريكة:** تقدر تغير الـ Orientation وأنت شغال في نص الملف لما تضيف صفحة جديدة: `doc.addPage({ layout: 'landscape' });` .

4. **الـ Margins (الهوامش):**

- الهوامش هي "منطقة الأمان" اللي بتمنع الكلام إنه يلزق في الحافة أو يختفي وقت الطباعة.
- الديفولت هو **72 point** (بوصة واحدة) من كل ناحية.
- تقدر تصفرها لو عايز ترسم "Full Page" عن طريق `margin: 0` .

---

1. الـ `doc.text()` (الأساسيات والذكاء التلقائي)

أول حاجة لازم تعرفها إن الميثود دي ذكية جداً.

- **الوضعية (Positioning):** لو قلت `doc.text("Hello")` من غير ما تحدد (X, Y)، هو بيبدأ من آخر مكان وقف فيه. لو حددت `doc.text("Hello", 100, 100)` ، هو هيروح هناك "يغرز" الكلمة.
- **الـ Automatic Line Wrap:** لو النص طويل وعدى عرض الصفحة، الـ PDFKit أوتوماتيك بينزل سطر جديد. ده بيخليك تركز في المحتوى وهو يركز في "التنسيق".

---

2. الـ Options (ليفل التنسيق العالي)

لما تفتح الـ Object بتاع الـ options `{}` ، إنت كأنك شغال على برنامج Word بالكود:

- **الـ Alignment (المحاذاة):**
- `align: 'left' | 'center' | 'right' | 'justify'` .
  - **الزتونة:** الـ `justify` دي هي اللي بتخلي الفواتير شكلها "شيك" لأنها بتوزع المسافات بين الكلمات عشان السطور كلها تبدأ وتنتهي في نفس النقطة.
- **الـ Layout Control:**
- `width` : هينزل سطر، width بتحدد عرض "العلبة" اللي النص هيتكتب فيها. لو النص أكبر من الـ.
- `height` : بتحدد أقصى طول للنص.
- `ellipsis` : لو النص طويل بزيادة عن المساحة، بدل ما يختفي، حط "..." في الآخر (حركة صايعة جداً في العناوين).

---

3. الـ Styling (الشياكة)

بدل ما تقعد ترسم خط تحت الكلمة يدوي بالـ Vector، الـ PDFKit بيعملها لك بضغطة زرار:

- `underline: true` : بيحط خط تحت الكلام.
- `strike: true` : بيشطب على الكلام (ممتاز لو بتعمل "خصم" وعايز تشطب على السعر القديم).
- `oblique: true` : بيخلي الكلام "مايل" (Italic).
- `link` : تقدر تخلي الكلمة "Clickable" تودي لـ URL معين.

---

4. الـ Columns (نظام الجرائد)

أقوى ميزة في الصفحة دي هي الـ **Multi-column Layout**.

- **المفهوم:** تقدر تقول للمكتبة "اكتبي النص ده في عمودين أو تلاتة".
- **الكود:** `{ columns: 3, columnGap: 15 }` .
- **الذكاء:** النص لما بيخلص في العمود الأول، بينط لوحده للعمود التاني. دي "الخلاصة" لو بتعمل بروشور أو منيو مطعم.

---

5. الـ Advanced Typography (للمحترفين بس)

لو عايز تتحكم في "النَفَس" بتاع النص:

- `lineGap` : المسافة بين السطور. زودها عشان تريح عين اليوزر وهو بيقرأ.
- `paragraphGap` : المسافة اللي بتسيبها بعد ما تخلص فقرة كاملة وتدوس Enter.
- `indent` : بيسيب مسافة في أول سطر في الفقرة (زي الكتب الروايات).
- `characterSpacing` : لو عايز تباعد بين الحروف وبعضها عشان تدي شكل معين Design.

---

6. الـ Text Measurement (المسطرة)

ساعات بتحتاج تعرف هو السطر ده هياخد مساحة قد إيه قبل ما تكتبه (مثلاً عشان ترسم برواز حواليه).

- `doc.widthOfString("text")` : بتقولك العرض بالـ Points.
- `doc.heightOfString("text")` : بتقولك الطول بالـ Points.
- **ليه دي مهمة؟** دي "العمود الفقري" لبناء الجداول (Tables). إنت بتحسب عرض أكبر كلمة عشان تحدد عرض العمود.

---

1. الـ Links (الكباري اللي بتوصلك)

دي أشهر أنواع الـ Annotations. الـ PDFKit بيسمح لك تعمل نوعين من اللينكات:

- **External Links:** لو عايز اليوزر لما يدوس على كلمة "Visit Website" يروح لـ URL حقيقي.
- **Internal Links (Destinations):** لو عندك ملف كبير وعايز اليوزر لما يدوس على كلمة في "الفهرس" ينط لصفحة تانية جوه نفس الملف.

---

2. الـ Notes (الـ Sticky Notes)

تخيل إنك عايز تسيب "ملحوظة" لليوزر بس مش عايزها تبقى مكتوبة وسط الكلام عشان متلخبطش التصميم.

- **المفهوم:** الـ `doc.note()` بتعمل أيقونة صغيرة (زي أيقونة الكومنت) لما اليوزر يدوس عليها، يفتح له صندوق فيه نص طويل.
- **الزتونة:** دي ممتازة للـ **Feedback** أو لو عايز تشرح مصطلح تقني من غير ما تاخد مساحة في الورقة.

---

3. الـ Markup (التظليل والشطب)

دي الأدوات اللي كنا بنستخدمها وإحنا بنذاكر في الكتب، بس دلوقتي بالكود:

- **Highlight:** بيعمل "تظليل" فسفوري ورا النص.
- **Underline:** بيحط خط تحت الكلام (كـ Annotation مش كـ Text style).
- **Strikeout:** بيحط خط في نص الكلام (للشطب).
- **Squiggly:** بيعمل خط متعرج تحت الكلام.

---

4. الـ Bounding Box (برواز التفاعل)

دي أهم حتة تقنية في الـ Annotations. أي Annotation في الدنيا لازم يتحدد بـ **Rectangle** (مستطيل).

- **التحليل:** إنت بتقول للمكتبة: "في المساحة من (x1, y1) لـ (x2, y2)، لو اليوزر داس هنا.. نفذ كذا".
- **القوة:** إنت مش محشور في النص بس؛ إنت ممكن تعمل "مستطيل سحري" فوق صورة أو لوجو، بحيث لما اليوزر يدوس على الصورة يفتح لينك معين.

---

5. الـ Advanced Annotations (ليفل المحترفين)

الصفحة بتشير إنك تقدر تتحكم في خصائص تانية زي:

- **Color:** لون الأيقونة أو التظليل.
- **Author:** بيظهر في برامج الـ PDF زي Adobe Acrobat). مين اللي ساب الملحوظة دي
- **Creation Date:** تاريخ كتابة الـ Annotation.

---

6. الـ Navigation (الـ GoTo)

دي ميزة "صايعة" جداً في الـ PDFKit. تقدر تعمل `addNamedDestination` لنقطة معينة في الملف، وبعدين في أي مكان تاني تعمل لينك بيشاور على الـ Destination ده.

- **الاستخدام:** ده "العمود الفقري" لأي كتاب إلكتروني أو تقرير مالي طويل فيه فصول وأقسام.

1. الـ Interactive Forms (الفورم اللي بتنطق)

بدل ما اليوزر يطبع الورقة ويملأها بالقلم، إنت بتعمل له **AcroForms** يقدر يكتب فيها وهو فاتح اللابتوب.

- **المفهوم:** الـ PDFKit بيخليك تكريت "حقول" (Fields) تفاعلية.
- **أنواع الحقول:**
- `formText` : مكان يكتب فيه نص (زي الاسم أو العنوان).
- `formCheckbox` : مربع اختيار (صح أو غلط).
- `formCombo` : قائمة منسدلة يختار منها (Dropdown).
- **الزتونة:** كل حقل له اسم (`name`) وقيمة افتراضية (`value`). لما اليوزر يملى البيانات دي ويسيف، إنت ممكن بعد كدة "تشفط" الداتا دي من الملف برمجياً.

---

2. الـ Accessibility (الملف الصديق للكل)

دي الحتة اللي بتميز المبرمج "البريميم". الـ PDF بطبعه عبارة عن رسومات، والـ **Screen Readers** (برامج القراءة للمكفوفين) ساعات بتشوفه "أبيض يا ورد".

- **الـ Tagged PDF:** ده نظام بيخليك تحط "تاجات" (زي الـ HTML) ورا العناصر.
  - العنوان بياخد تاج `H1` .
  - الصورة بتاخد `Alt Text` .
- **تفعيل الخاصية:** لازم وأنت بتعرف الـ `doc` تقوله إن الملف ده **Tagged**: JavaScript

```
;const doc = new PDFDocument({ pdfVersion: '1.5', tagged: true })
```

- **ليه دي مهمة؟** في دول كتير، لو بتعمل ملفات لجهات حكومية أو شركات دولية، القانون بيلزمك إن الملف يكون **Accessible** عشان يضمنوا إن كل فئات المجتمع تقدر توصله.

---

3. الـ Tables (المعضلة الكبرى)

هنا بقى فيه "صدمة" صغيرة؛ **PDFKit مفيهاش ميثود `doc.table()` جاهزة!**

- **المشكلة:** الـ PDFKit مكتبة "Low-level"، يعني بتديك القلم والمسطرة وإنت اللي ترسم.
- **الحل اليدوي:** عشان تعمل جدول، إنت محتاج تعمل "ميكس" بين اللي اتعلمناه:

1. **Vector Graphics:** ترسم الخطوط العرضية والطولية بـ `lineTo` .
2. **Text:** تحسب مكان الكلام بالظبط عشان يجي "في نص الخانة" باستخدام `widthOfString` .

- **الحل الصايع:** المطورين عملوا مكتبات مساعدة بتركب فوق PDFKit (زي `pdfkit-table`) بتاخد منك JSON وتطلع لك جدول بضغطة زرار.

```
const PDFDocument = require('pdfkit');
const fs = require('fs');
const path = require('path');

exports.generateMassiveInvoice = (req, res, next) => {
    const doc = new PDFDocument({
        size: 'A4',
        margins: { top: 50, left: 50, right: 50, bottom: 50 },
        info: { Title: 'Massive Invoice', Author: 'Gemini Store' }
    });

    // 1. Streaming: بالـ "الماسورة" وصل Response فوراً
    res.setHeader('Content-Type', 'application/pdf');
    res.setHeader('Content-Disposition', 'inline; filename="massive-
invoice.pdf"');
    doc.pipe(res);

    // --- 2. Vector Graphics (البرواز واللوجو) ---
    // رسم برواز خارجي للصفحة
    doc.rect(20, 20, 555, 780).lineWidth(2).stroke('#333');

    // رسم مثلث ديكوري في الركن (Vector Path)
    doc.moveTo(500, 20).lineTo(575, 20).lineTo(575, 95).fill('#007bff');

    // --- 3. Images (اللوجو) ---
    // ملحوظة: تأكد إن عندك صورة في المسار ده أو استبدله بمسار حقيقي
    // doc.image('path/to/logo.png', 50, 45, { width: 50 });

    // --- 4. Text & Typography (العناوين) ---
    doc.fillColor('#444444')
       .fontSize(20)
       .text('GEMINI E-SHOP', 110, 57)
       .fontSize(10)
       .text('123 Cairo Street, Egypt', 110, 80)
       .text('Phone: +20 123 456 789', 110, 95)
       .moveDown();

    // خط فاصل (Vector Line)
    doc.moveTo(50, 120).lineTo(550, 120).lineWidth(1).stroke('#ccc');

    // --- 5. Layout & Text (بيانات العميل) ---
    doc.fillColor('#000')
       .fontSize(12)
       .text(`Invoice Number: #INV-${Math.floor(Math.random() * 10000)}`, 50,
```

```
140)
       .text(`Date: ${new Date().toLocaleDateString()}`, 50, 155)
       .text('Customer: Ahmed Mohamed', 50, 170)
       .moveDown();

    // --- 6. The Table (هندسة الجداول يدوياً) ---
    const tableTop = 230;
    const itemCodeX = 50;
    const descriptionX = 150;
    const quantityX = 350;
    const priceX = 420;
    const totalX = 500;

    // Header بتاع الجدول
    doc.fillColor('#007bff')
       .fontSize(12)
       .text('Code', itemCodeX, tableTop)
       .text('Description', descriptionX, tableTop)
       .text('Qty', quantityX, tableTop)
       .text('Price', priceX, tableTop)
       .text('Total', totalX, tableTop);

    doc.moveTo(50, tableTop + 15).lineTo(550, tableTop + 15).stroke();

    // بيانات المنتجات (Loop)
    const items = [
        { code: 'A1', desc: 'Node.js Mastery Course', qty: 1, price: 50 },
        { code: 'B2', desc: 'PDFKit Guide (E-book)', qty: 2, price: 15 },
        { code: 'C3', desc: 'Full-stack License', qty: 1, price: 100 }
    ];

    let i = 0;
    items.forEach(item => {
        const y = tableTop + 30 + (i * 25);
        doc.fillColor('#000')
           .fontSize(10)
           .text(item.code, itemCodeX, y)
           .text(item.desc, descriptionX, y)
           .text(item.qty, quantityX, y)
           .text(`$${item.price}`, priceX, y)
           .text(`$${item.qty * item.price}`, totalX, y);
        i++;
    });

    // --- 7. Total Calculation ---
    const subtotalY = tableTop + 40 + (i * 25);
```

```
    doc.moveTo(350, subtotalY).lineTo(550, subtotalY).stroke();
    doc.fontSize(12).text('Grand Total:', 350, subtotalY + 10);
    doc.fontSize(14).fillColor('#d9534f').text('$180.00', 500, subtotalY +
10);

    // --- 8. Annotations (روابط تفاعلية) ---
    doc.fillColor('#007bff')
       .text('Visit our website for support', 50, 700, {
           link: 'https://google.com',
           underline: true
       });

    // --- 9. Forms (حقل تفاعلي لليوزر) ---
    doc.fillColor('#000').text('Add Note:', 50, 730);
    doc.formText('userNote', 110, 725, 400, 20, {
        multiline: false,
        backgroundColor: [0.9, 0.9, 0.9]
    });

    // --- 10. End: قفل المحبس ---
    doc.end();
};
```

---

2. تشريح اللي حصل في الكود ده (The Breakdown):

1. **الـ Streaming:** استخدمنا `doc.pipe(res)` ؛ ده معناه إن الملف مش بيتخزن عندك ع السيرفر، هو بيتبعت "طازة" لليوزر حتة بحتة.
2. **الـ Vector Graphics:** رسمنا المثلث اللي فوق والبرواز والخطوط اللي تحت العناوين عشان ندي للفاتورة شكل "رسمي".
3. **الـ Coordinate System:** لو لاحظت، إحنا محددين الـ `x` والـ `y` لكل حتة (مثلاً `itemCodeX = 50`). ده اللي بيضمن إن الأعمدة نازلة مسطرة تحت بعضها.
4. **الـ Manual Table:** بما إن PDFKit مفيهاش جداول، إحنا عملنا Loop على الـ `items` وبنحسب الـ `y` لكل سطر (`tableTop + 30 + (i * 25)`).
5. **الـ Interactive Forms:** حطينا `formText` في الآخر. لو اليوزر فتح الملف ده ببرنامج Adobe Acrobat، هيقدر يكتب ملحوظة جوه المستطيل الرمادي.
6. **الـ Metadata:** الملف ده لو حملته وفتحت الـ Properties بتاعته، هتلاقي مكتوب فيه إن الـ Author هو "Gemini Store".